

# Real Estate AI Agent Module Documentation

---

## Real Estate AI Recommender Module Documentation

**Version:** 9.5 (Strategic Exploration)

**Architecture:** FAISS + XGBoost Two-Stage Retrieval & Ranking

**Primary Objective:** To deliver highly personalized, mathematically rigorous property recommendations by balancing semantic relevance, geographic constraints, and physical diversity.

### 1. System Architecture Overview

The `2_Recommender` module is a high-performance machine learning pipeline. It is not a simple database filter; it is a sophisticated AI engine that "learns" what makes a good property recommendation by simulating human behaviors.

The architecture follows a strict **Two-Stage Retrieval & Ranking** paradigm:

1. **Retrieval (FAISS):** Cast a wide net to find ~300 semantically similar properties.
2. **Ranking (XGBoost):** Use an engineered stack of mathematical features to score and re-rank those properties into a flawless Top 10 list.

### Core Modules

- `1_config_and_cache.py`: Global state, caching utilities, and hyperparameter thresholds.
- `2_math_and_features.py`: The vectorization engine (Haversine distances, Jaccard similarities, Feature Stacking).
- `3_teacher.py`: The Behavioral Simulator that generates ground-truth labels by acting as different buyer personas.
- `4_main_orchestration.py`: The master script that trains the entire system from scratch.
- `6_query_adapter.py`: The production inference adapter that bridges the LangGraph Agent to the trained XGBoost model.

### 2. Configuration & Caching (`1_config_and_cache.py`)

This file holds the absolute "laws of physics" for the Recommender system to prevent magic numbers from bleeding into the logic.

## 2.1 Thresholds & Hyperparameters

- **`geo\_fence\_km` (25):** The maximum allowable radius before the ranking model applies an exponential penalty to the property's score.
- **`budget\_buffer` (0.30):** Allows the model to consider properties up to 30% over the user's stated budget if the quality is exceptionally high (Premium Upselling).
- **`mmr\_lambda` (0.85):** Controls the tradeoff between Relevance and Diversity. 1.0 means perfect relevance (ignoring diversity), while 0.85 forces the model to slightly penalize properties that are physically located right next to each other.

## 2.2 Property Similarity Matrix

A hardcoded heuristic dictionary that maps the architectural similarity of different units. For example, a `Villa` and `Townhouse` share an `0.8` similarity score, preventing the model from severely penalizing a user who asked for a `Villa` but is shown a sprawling `Townhouse`.

## 2.3 Caching Layer

Uses `joblib` to serialize massive numpy matrices (`save_cache`, `load_cache`). This allows the orchestration pipeline to be paused and resumed without recalculating 10,000 text embeddings.

## 3. Mathematical Feature Engineering (`2\_math\_and\_features.py`)

The XGBoost model cannot "read" text; it only understands numbers. This module is responsible for converting raw property data into an extremely dense (`N`, `16`) numpy matrix.

### 3.1 The Feature Store

A RAM-optimized class that pre-calculates the `amenity_matrix` for the entire catalog. When a query comes in, it uses `get_amenity_matrix_batch()` to retrieve hundreds of vectors in  $O(1)$  time for lightning-fast comparisons.

### 3.2 Engineering Functions

- **`haversine\_vectorized()`:** Calculates the curved-earth distance (in kilometers) between arrays of GPS coordinates using radians.
- **`calculate\_jaccard()`:** Computes the Intersection-over-Union between the query's requested amenities (e.g., Balcony, Pool) and the candidate's actual amenities.
- **`mmr\_rerank()`:** The Maximal Marginal Relevance algorithm. After XGBoost scores the properties, this algorithm iterates through them, applying a penalty if a highly-scored property is physically too close to a property that was already selected. This guarantees geographic diversity in the final Top 10.

### 3.3 The Feature Engine Stack

The `build_feature_stack()` function is the heart of the model. It guarantees that Training and Production Inference use the exact same logic. It engineers 16 features, including:

- `price_proximity_log`: How close the property price is to the budget.
- `asymmetric_budget_penalty`: Applies a mathematical log-penalty ONLY if the property exceeds the budget.
- `market_premium_ratio`: Calculates if the property is overpriced relative to its specific town's median price per square foot.

#### 4. The Persona Teacher (`3\_teacher.py`)

Machine Learning rankers require  $y$  (Ground Truth) to learn. Instead of relying on years of historical user click-data, this module uses **Synthetic Persona Generation** to teach the XGBoost model what a "good" recommendation looks like.

##### 4.1 The Archetypes

It simulates 7 distinct human personalities (e.g., "The Bargain Hunter", "The Luxury Elite", "The Hidden Gem Hunter"). Each persona has a unique set of `weights`:

- **\*Bargain Hunter:** Cares 80% about Price, 0% about size.
- **\*Family Weaver:** Cares 40% about Geo (schools/location), 30% about Size.

##### 4.2 Label Generation (`get\_spm\_labels`)

For a given query and set of candidates, the Teacher:

3. Calculates raw component scores (Price, Geo, Size, Amenities) normalized from 0 to 1.
4. Applies the selected Persona's weights.
5. Applies strict Domain Bonuses (e.g., Luxury Elite personas heavily reward properties with a high `listing_quality_score`).
6. Injects simulated human noise (`np.random.normal(0, 0.02)`) to prevent the model from overfitting to a rigid formula.
7. Maps the continuous score into a discrete class label:
  - 4: Elite Match
  - 3: Strong Match
  - 2: Relevant Match
  - 1: Weak/Discovery Match
  - 0: Total Mismatch (e.g., Rent vs Buy crossover)

#### 5. Main Orchestration Pipeline (`4\_main\_orchestration.py`)

This file is a massive, automated script that builds the entire brain of the Recommender from scratch through a 5-stage funnel.

- **Stage 1: Neural Encoding:** Uses `paraphrase-multilingual-MiniLM-L12-v2` to read property descriptions and convert them into 384-dimensional dense vectors.
- **Stage 2: FAISS Indexing:** Loads the vectors into an `IndexFlatIP` (Inner Product) search index for instantaneous semantic similarity lookups.

- **Stage 3: Training Funnel:** Simulates 400 user queries. For each query, it pulls 300 Semantic Candidates (via FAISS) and 50 Random Exploratory Candidates. It feeds these 350 properties into the `FeatureEngine` and asks the `Teacher` to grade them.
- **Stage 4: XGBoost Training:** Feeds the millions of engineered data points into an `XGBRanker`. It trains to minimize the `ndcg` loss function across groups (queries), learning exactly how to weight price, location, and amenities based on the `Teacher`'s lessons.
- **Stage 5: Evaluation:** Runs a blind test on 100 hold-out queries. It prints a Scoreboard containing:
  - **nDCG@10:** Overall ranking quality.
  - **MRR:** How often a perfect "Level 4" match appears as the #1 result.
  - **Spatial Diversity:** Average distance between recommended properties.

## 6. Adapters & Interfaces

### 6.1 The Query Adapter (``6_query_adapter.py``)

The production interface for the LangGraph Agent.

When the Agent sends a JSON dictionary (e.g., `{ "town": "Zayed", "price_max": 15000000 }`), this file:

8. Generates a "Synthetic Row" (faking a real property based on the user's constraints).
9. Generates an ad-hoc neural embedding for the text components.
10. Queries FAISS for the Top 300 baseline matches.
11. Passes the matches through the `FeatureEngine` to generate the 16-column matrix.
12. Pushes the matrix through the trained XGBoost model to get predicted scores.
13. Runs `mmr_rerank()` to ensure the top 10 results aren't all clustered in the exact same compound.

### 6.2 The Export Adapter (``5_export_adapter.py``)

Acts as an Anti-Corruption Layer. It grabs the highly cleaned and engineered dataset produced by the Recommender's internal preprocessor and writes it out as `Rec_To_Ana_Catalog.parquet`. This guarantees that the downstream `3.Analyzer` module is analyzing the exact same data the Recommender was trained on, preventing logical drift between subsystems.